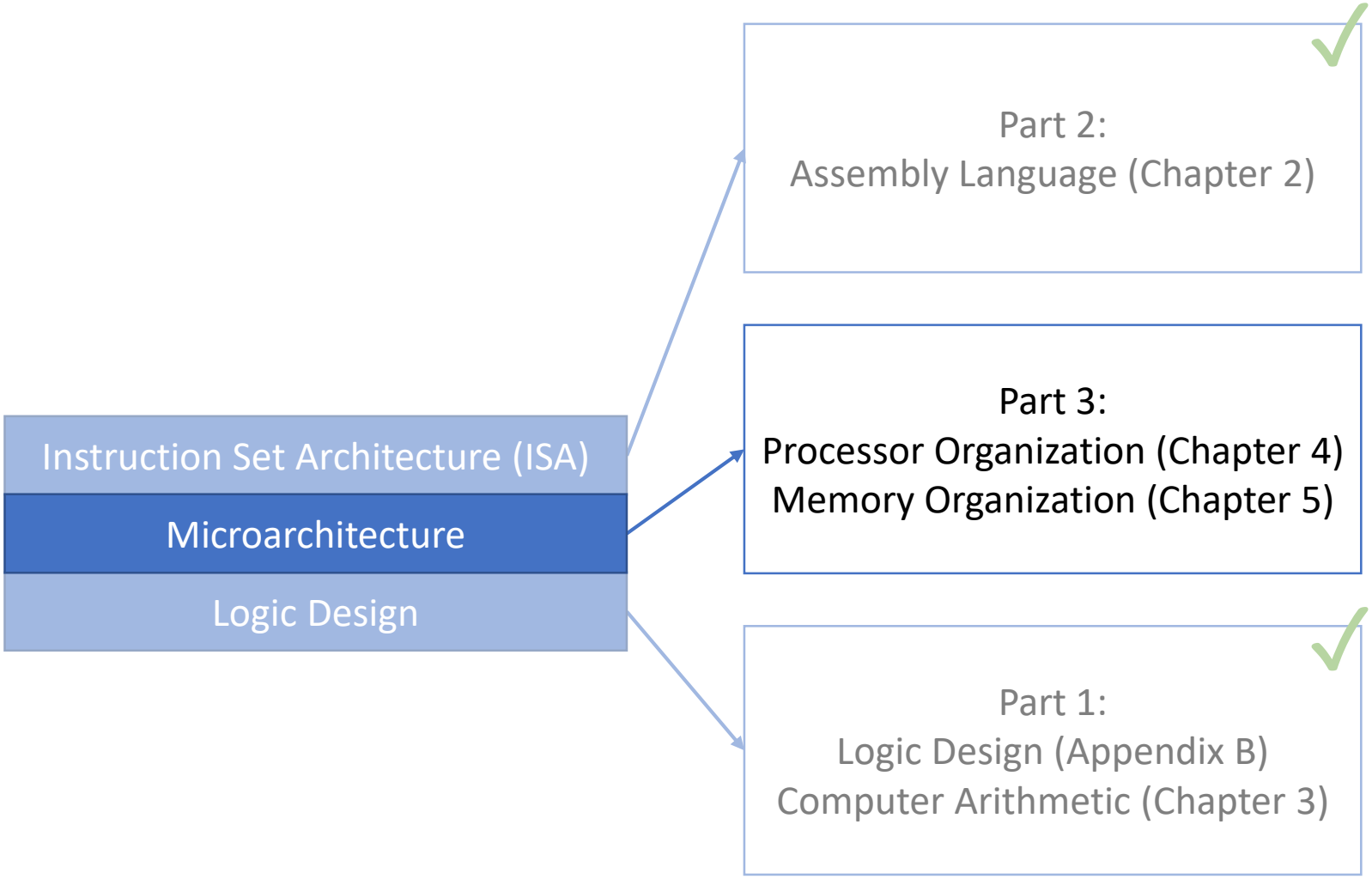
Lecture 27: Instruction-Level Parallelism

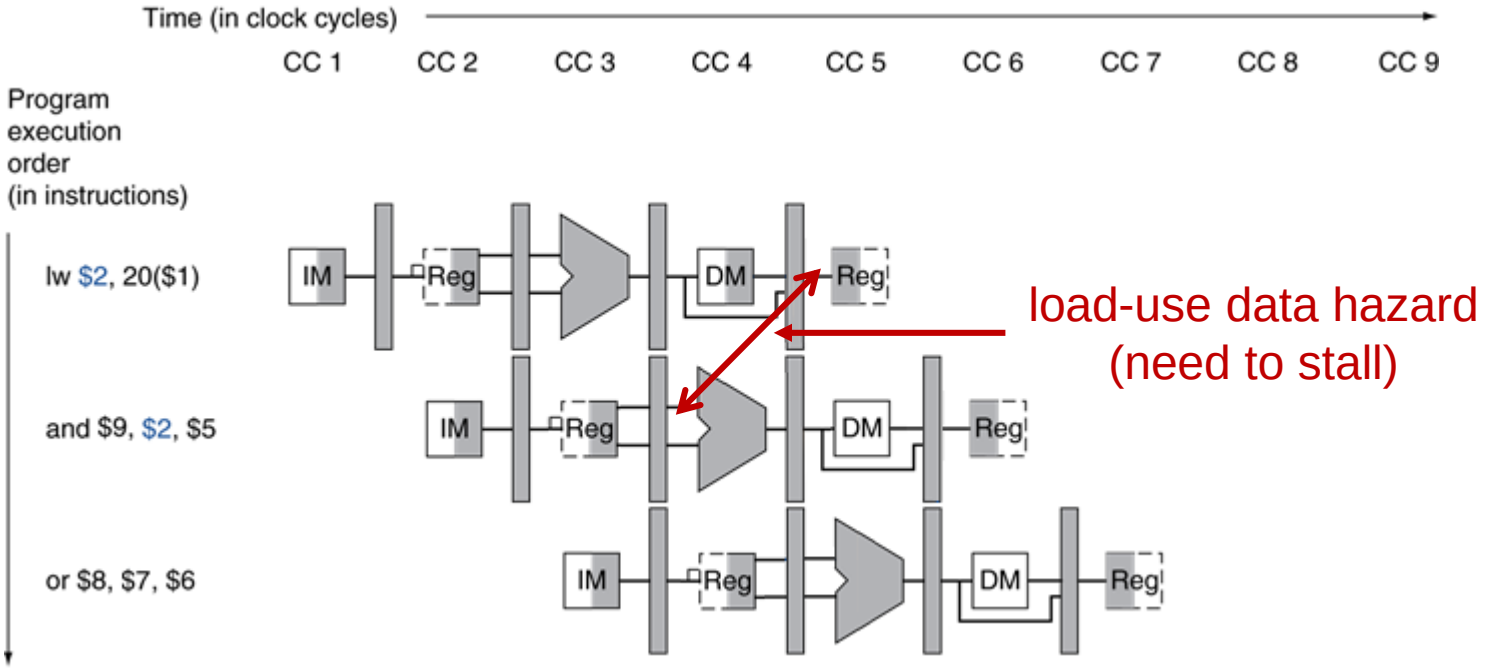
CMPS 221 – Computer Organization and Design

Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

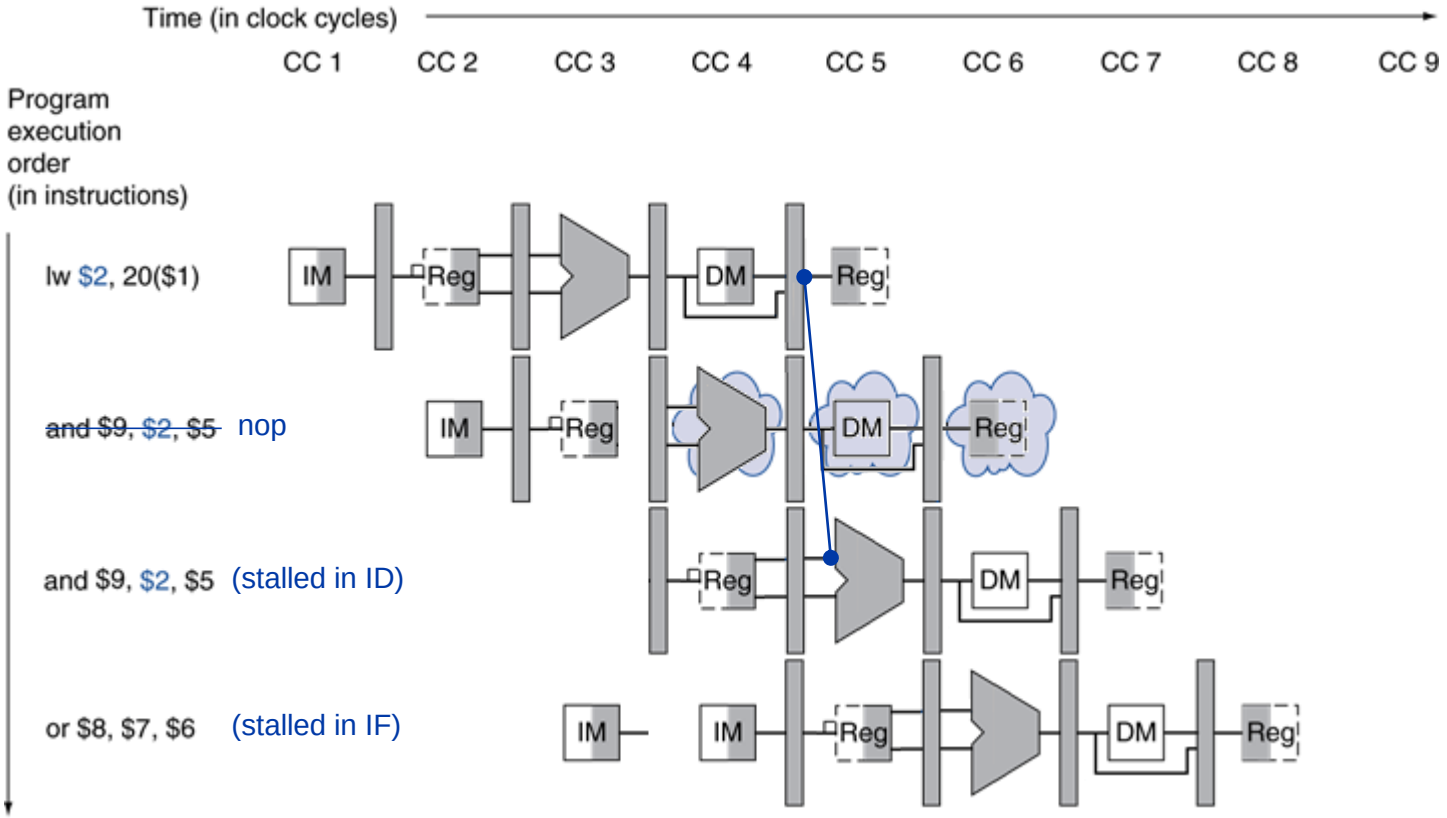
Last Time: Stalling, Branch Data Hazards



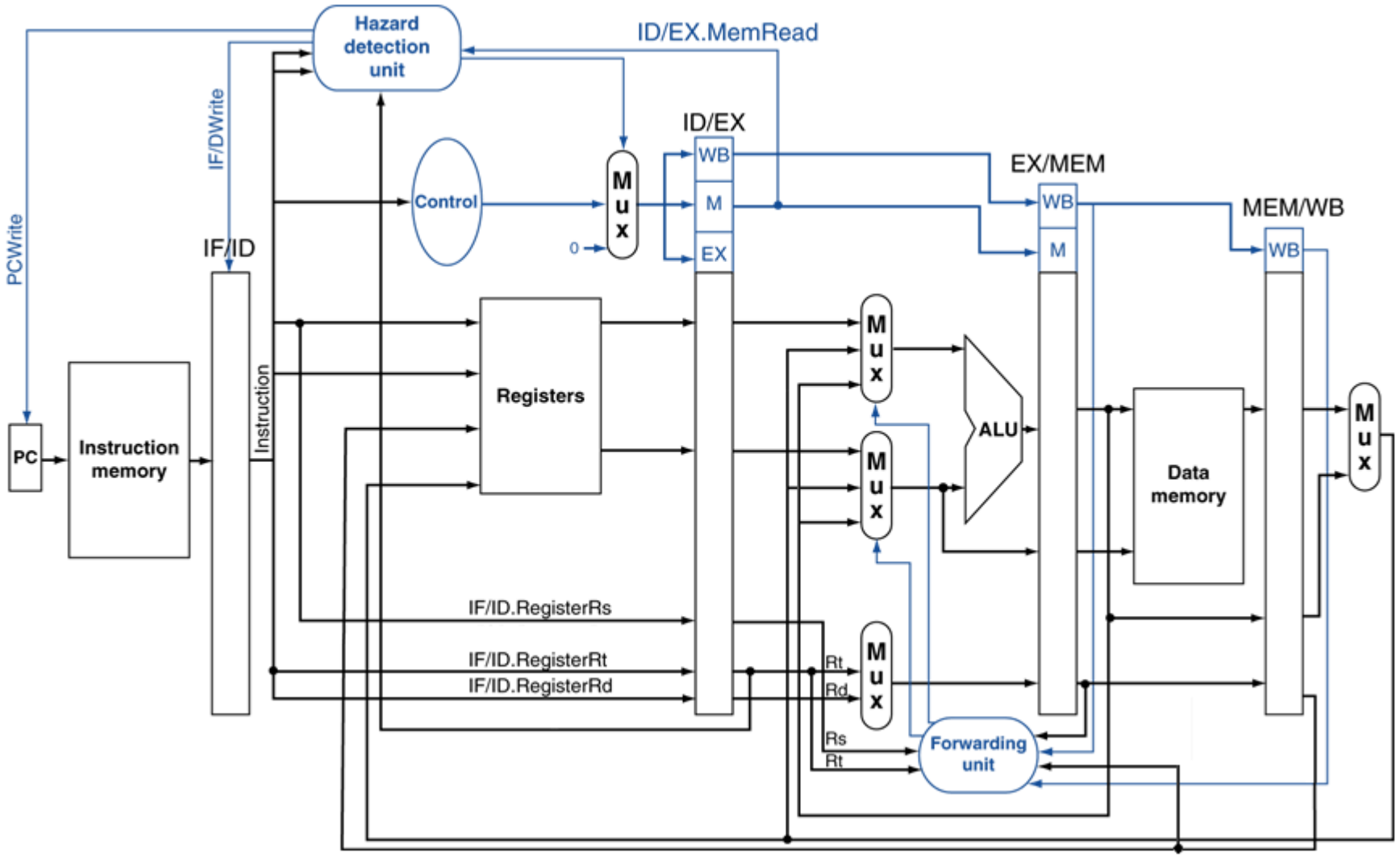
Load-Use Data Hazard



Load-Use Data Hazard

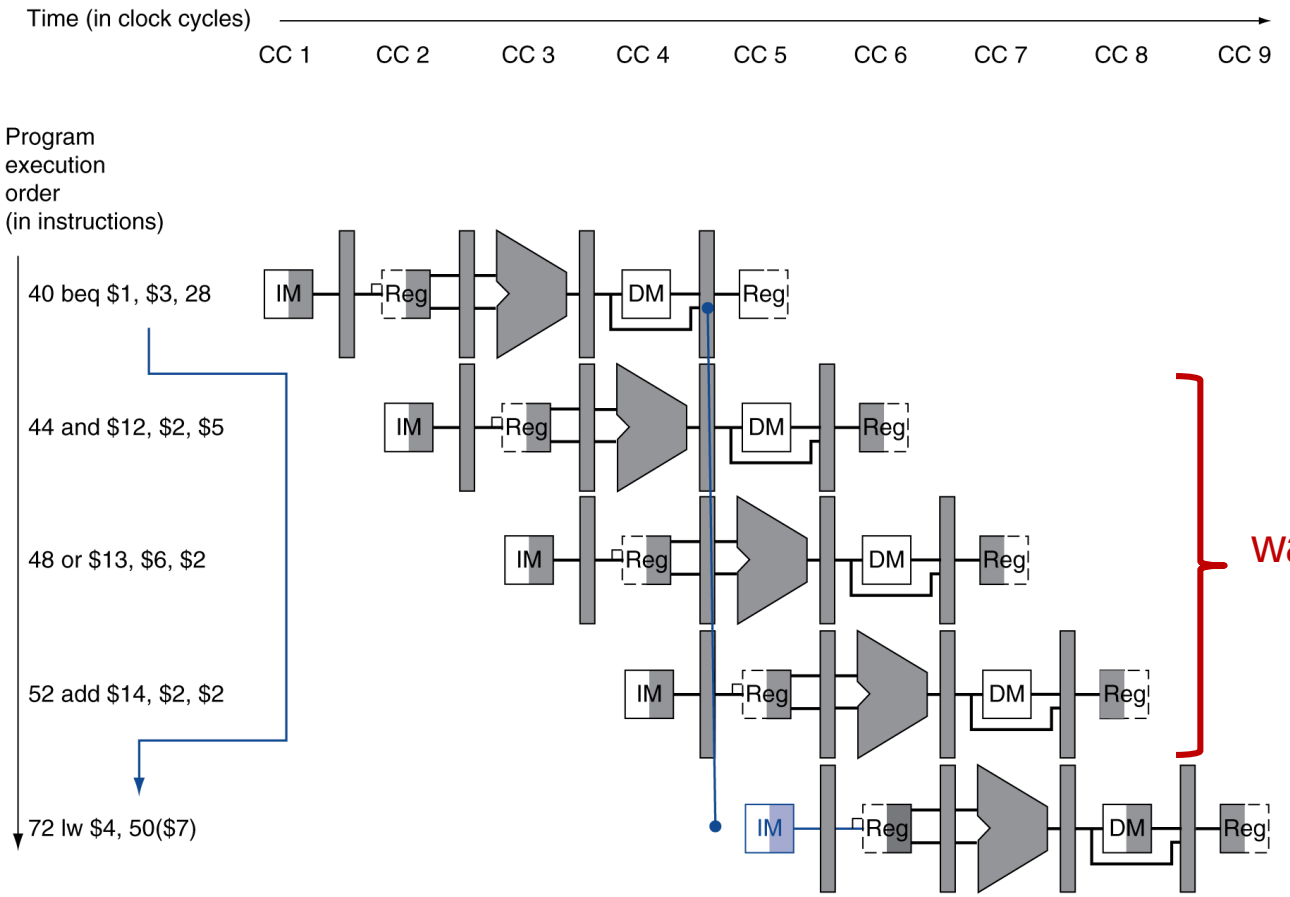


Hazard Detection Unit



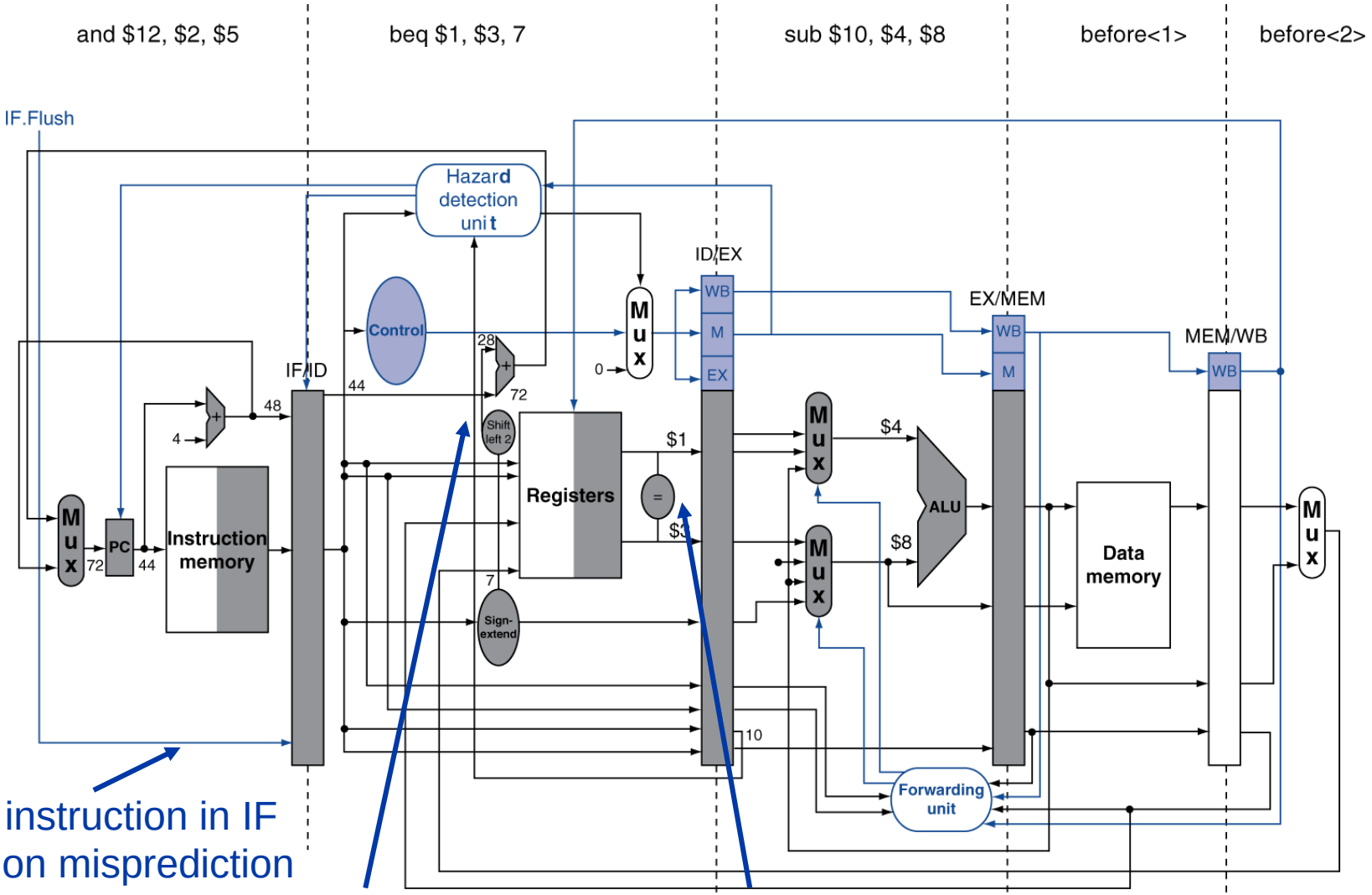
Branch Hazards

- If branch outcome determined in MEM



Recall: Solution was to resolve branch at the end of the ID stage

Reducing Branch Delay



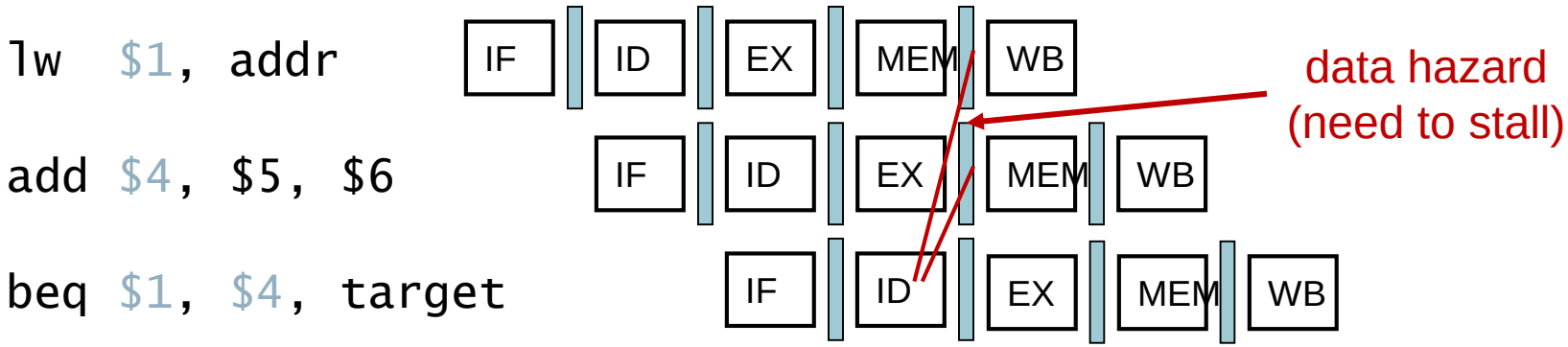
flush instruction in IF stage on misprediction

calculate branch target in ID stage

compare registers in ID stage

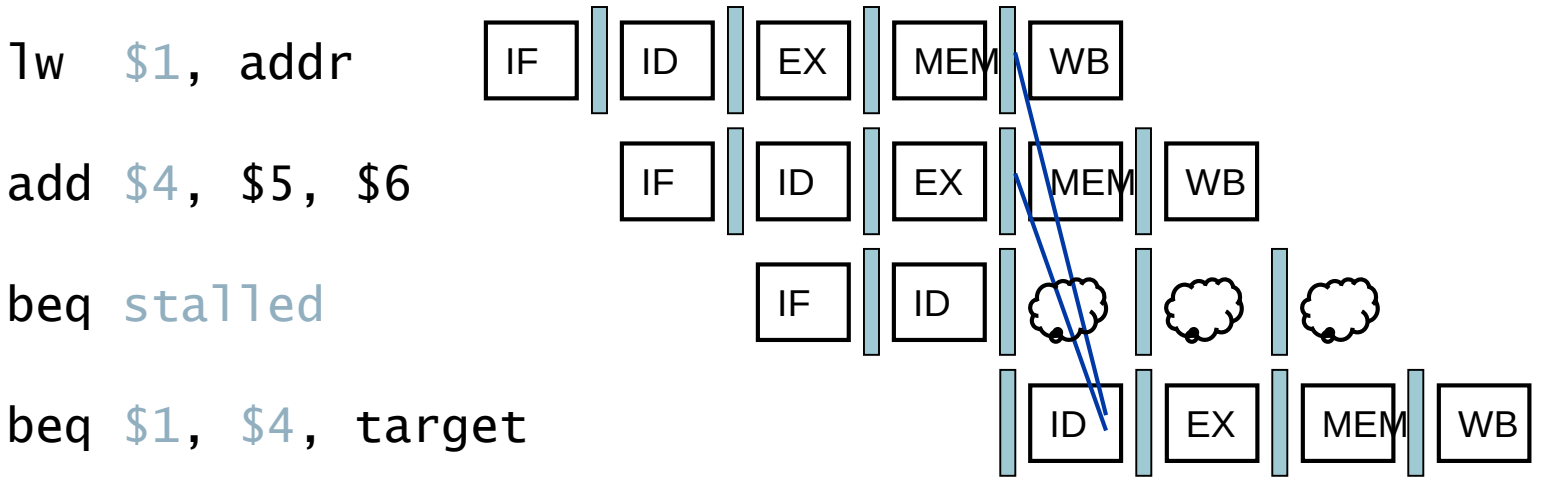
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction



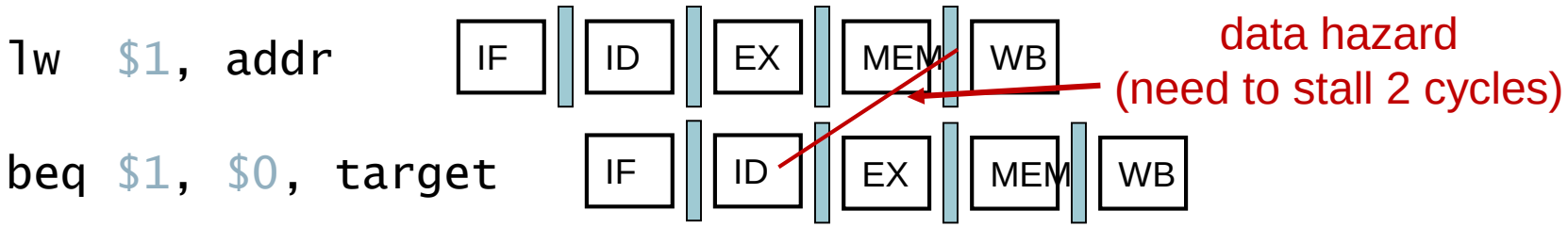
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction



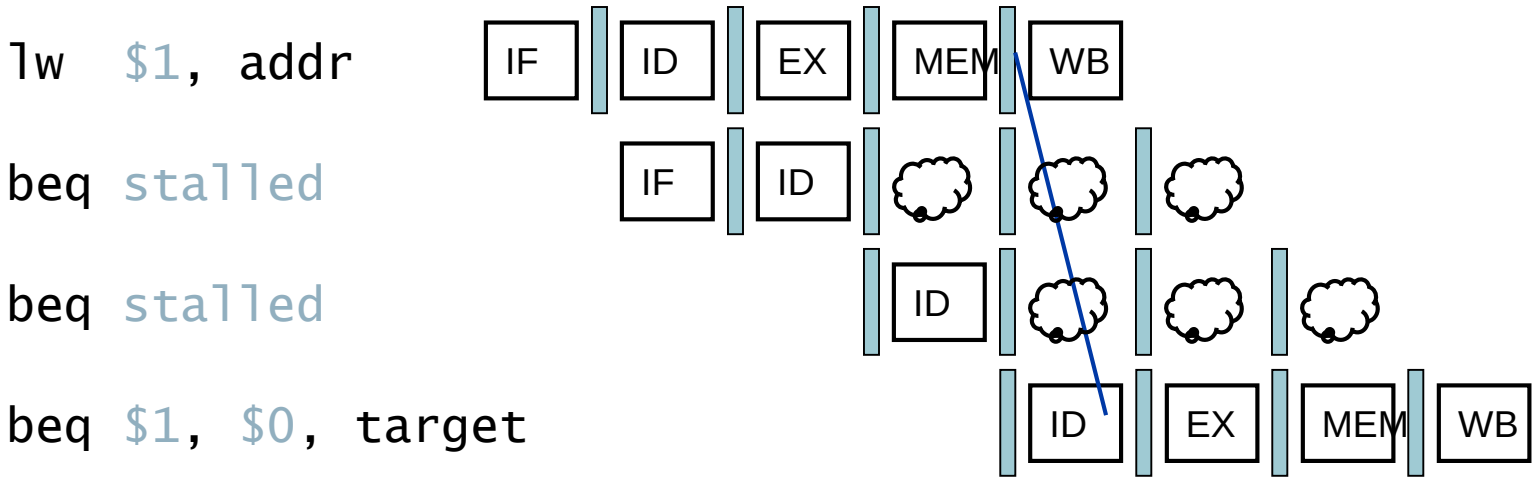
Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction

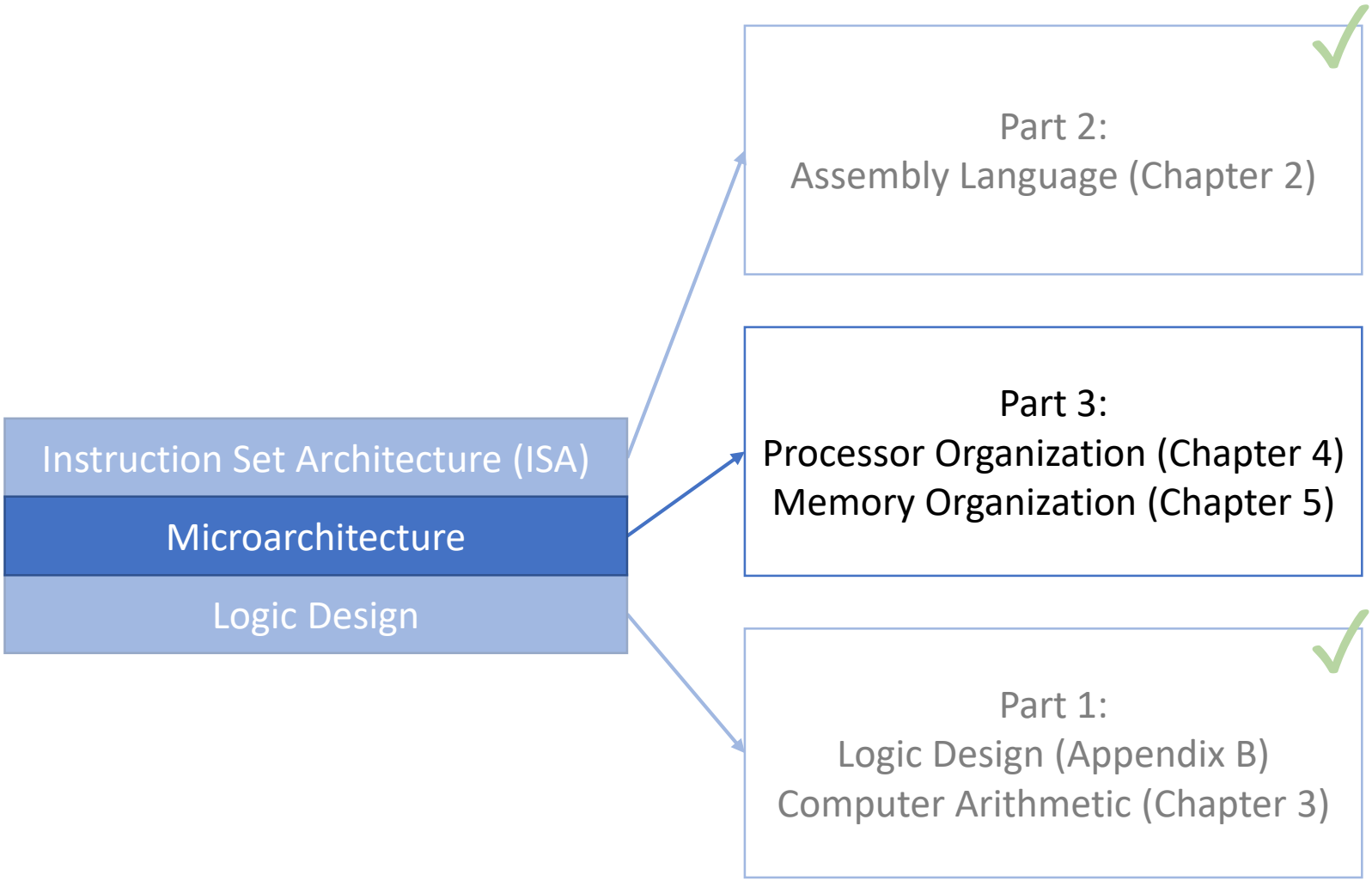


Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction



Today: Instruction-Level Parallelism



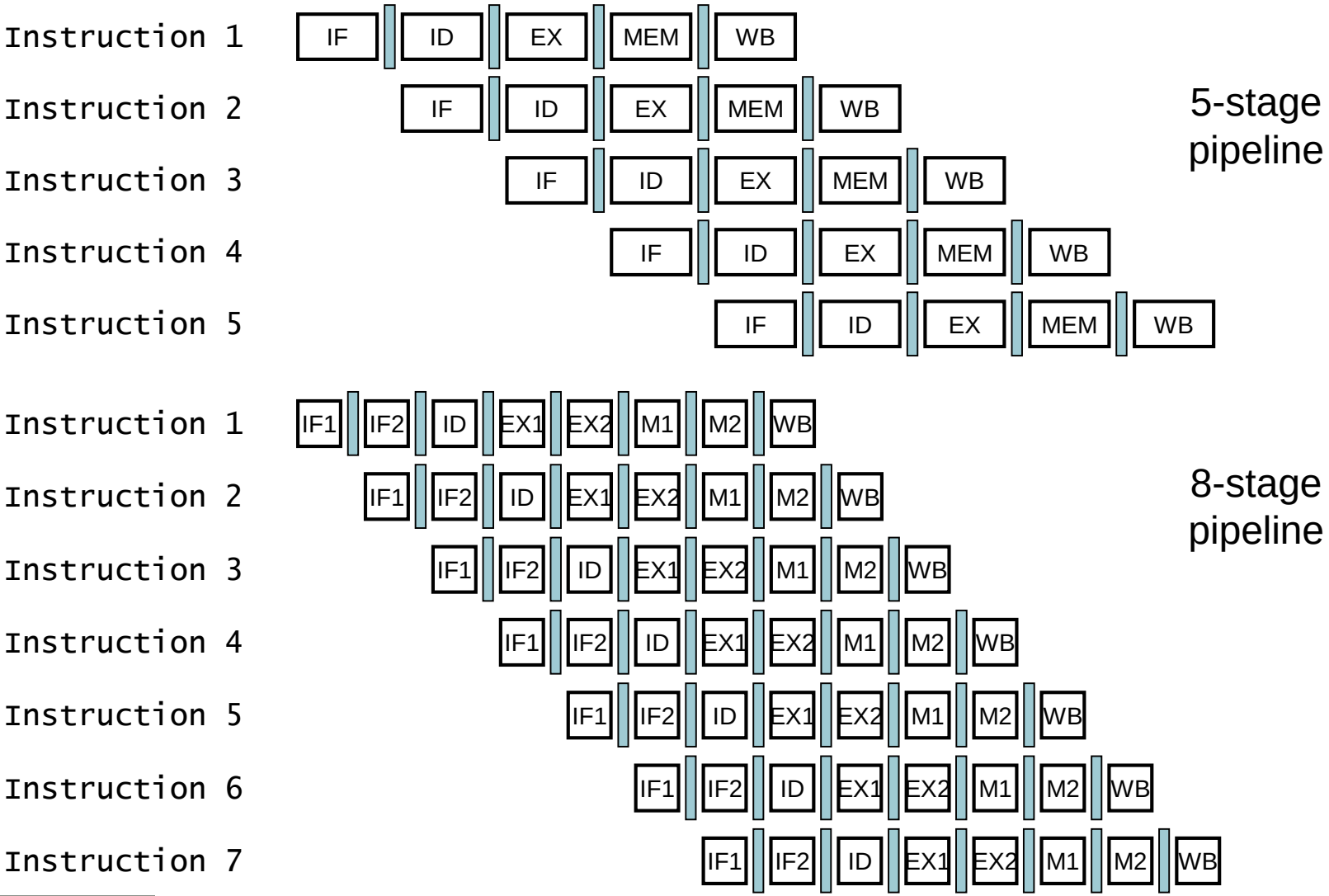
Instruction-Level Parallelism

- **Instruction-level parallelism** (ILP) refers to extracting parallelism from among different instructions in the same sequence
- We have already extracted ILP via pipelining
  - Multiple instructions execute simultaneously in different stages
- How to extract more ILP?
  - Make pipelines *deeper*
  - Make pipelines *wider*

Making Pipelines Deeper

- Recall: Previous pipeline had five unbalanced stages with a 200ps clock cycle:
  - 100ps for ID and WB
  - 200ps for IF, EX, and MEM
- Assume a deeper pipeline is used:
  - Break IF, EX, and MEM into two 100ps stages each
  - The resulting pipeline has eight balanced stages with 100ps clock cycle

Deep Pipeline Multi-cycle Diagram

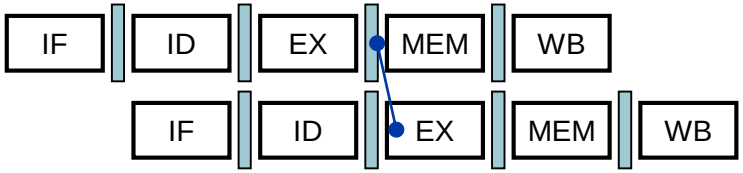


Shallow vs. Deep Pipelines

- Recall: For the five-stage unbalanced pipeline:
  - $t_{nonpipelined} = N \cdot 800ps$
  - $t_{pipelined} = (N + 5 - 1) \cdot 200ps$
  - $speedup = \frac{N \cdot 800ps}{(N+5-1) \cdot 200ps} \xrightarrow{large\ N} 4\times$
- For the eight-stage balanced pipeline:
  - $t_{pipelined} = (N + 8 - 1) \cdot 100ps$
  - $speedup = \frac{N \cdot 800ps}{(N+8-1) \cdot 100ps} \xrightarrow{large\ N} 8\times$

Deep Pipeline Hazards

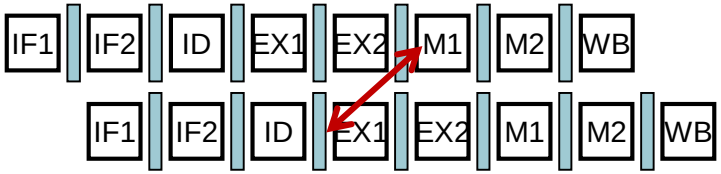
```
add $t0, $s0, $s1
sub $v0, $t0, $t1
```



5-stage pipeline

Forwarding from MEM to EX avoids stalls

```
add $t0, $s0, $s1
sub $v0, $t0, $t1
```

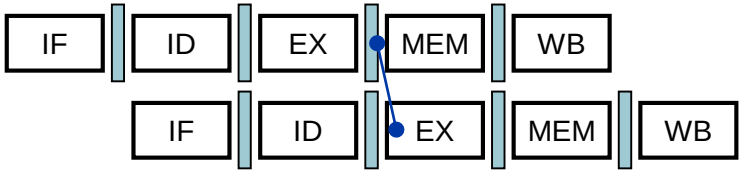


8-stage pipeline

EX now takes two stages so its result is not available in the next cycle

Deep Pipeline Hazards

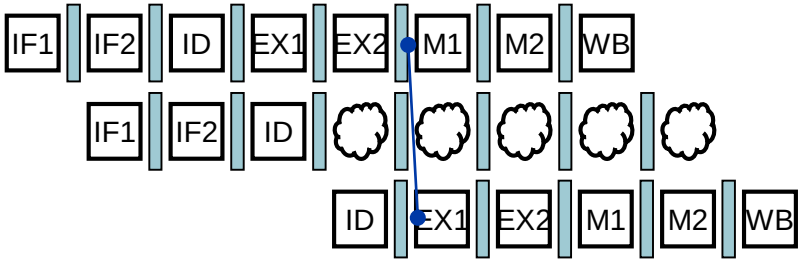
```
add $t0, $s0, $s1
sub $v0, $t0, $t1
```



5-stage pipeline

Forwarding from MEM to EX avoids stalls

```
add $t0, $s0, $s1
~~sub $v0, $t0, $t1~~
sub $v0, $t0, $t1
```

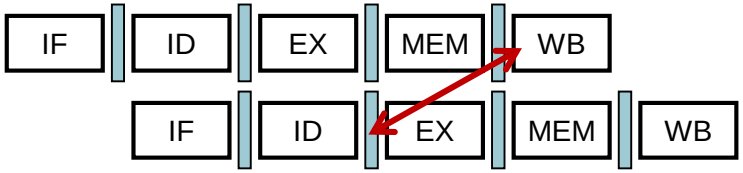


8-stage pipeline

Must stall until EX2 completes to forward result

Deep Pipeline Load-Use Hazards

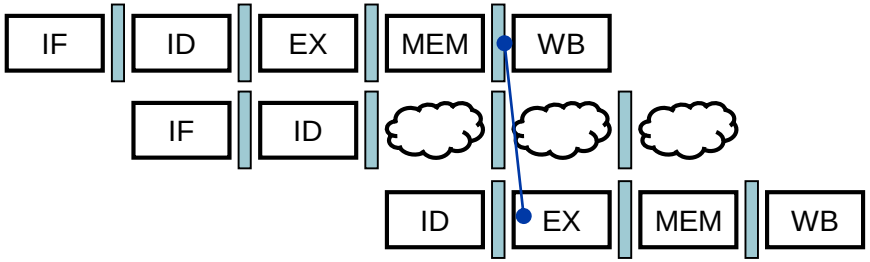
```
lw  $t0, 0($s0)
sub $v0, $t0, $t1
```



5-stage pipeline

Deep Pipeline Load-Use Hazards

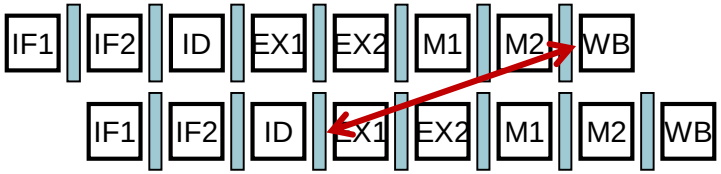
```
lw  $t0, 0($s0)
sub $v0, $t0, $t1
sub $v0, $t0, $t1
```



5-stage pipeline

Need to stall once on load-use data hazard

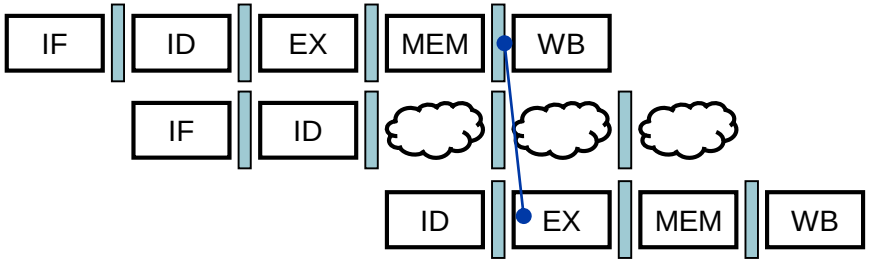
```
lw  $t0, 0($s0)
sub $v0, $t0, $t1
```



8-stage pipeline

Deep Pipeline Load-Use Hazards

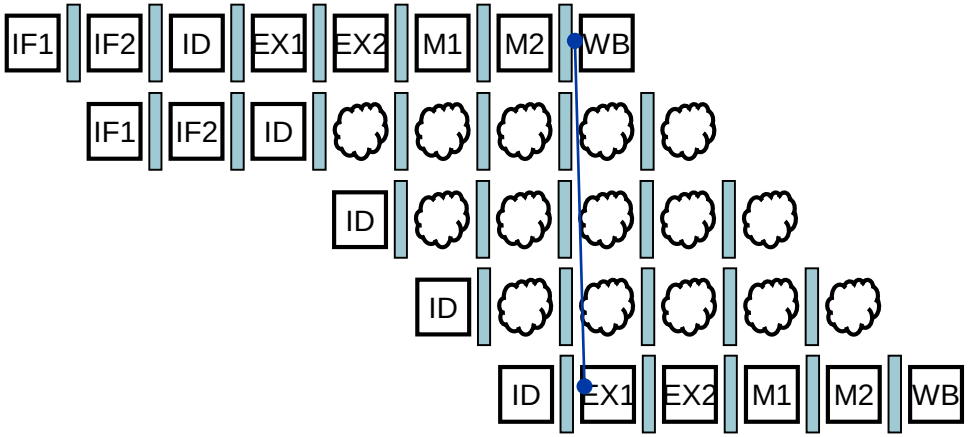
```
lw  $t0, 0($s0)
sub $v0, $t0, $t1
sub $v0, $t0, $t1
```



5-stage pipeline

Need to stall once on load-use data hazard

```
lw  $t0, 0($s0)
sub $v0, $t0, $t1
sub $v0, $t0, $t1
sub $v0, $t0, $t1
sub $v0, $t0, $t1
```



8-stage pipeline

Need to stall three times on load use data hazards

Deep Pipelines Tradeoffs

- Advantage: deep pipelines provide higher speedup by extracting more ILP
- Disadvantage: deep pipelines suffer from more hazards and stalls when the code does not exhibit a sufficient amount of ILP
  - We will see how to fill these stalls later
  - Typical pipeline depth is 14 stages in a modern processor

Making Pipelines Wider

- Recall: Previous pipeline fetched and executed one instruction in each stage
- A wider pipeline can fetch and execute multiple instructions in each stage
  - Known as **multiple issue**
  - Need to replicate pipeline stages
- Challenge: How to identify instructions that can be issued together?

Multiple Issue

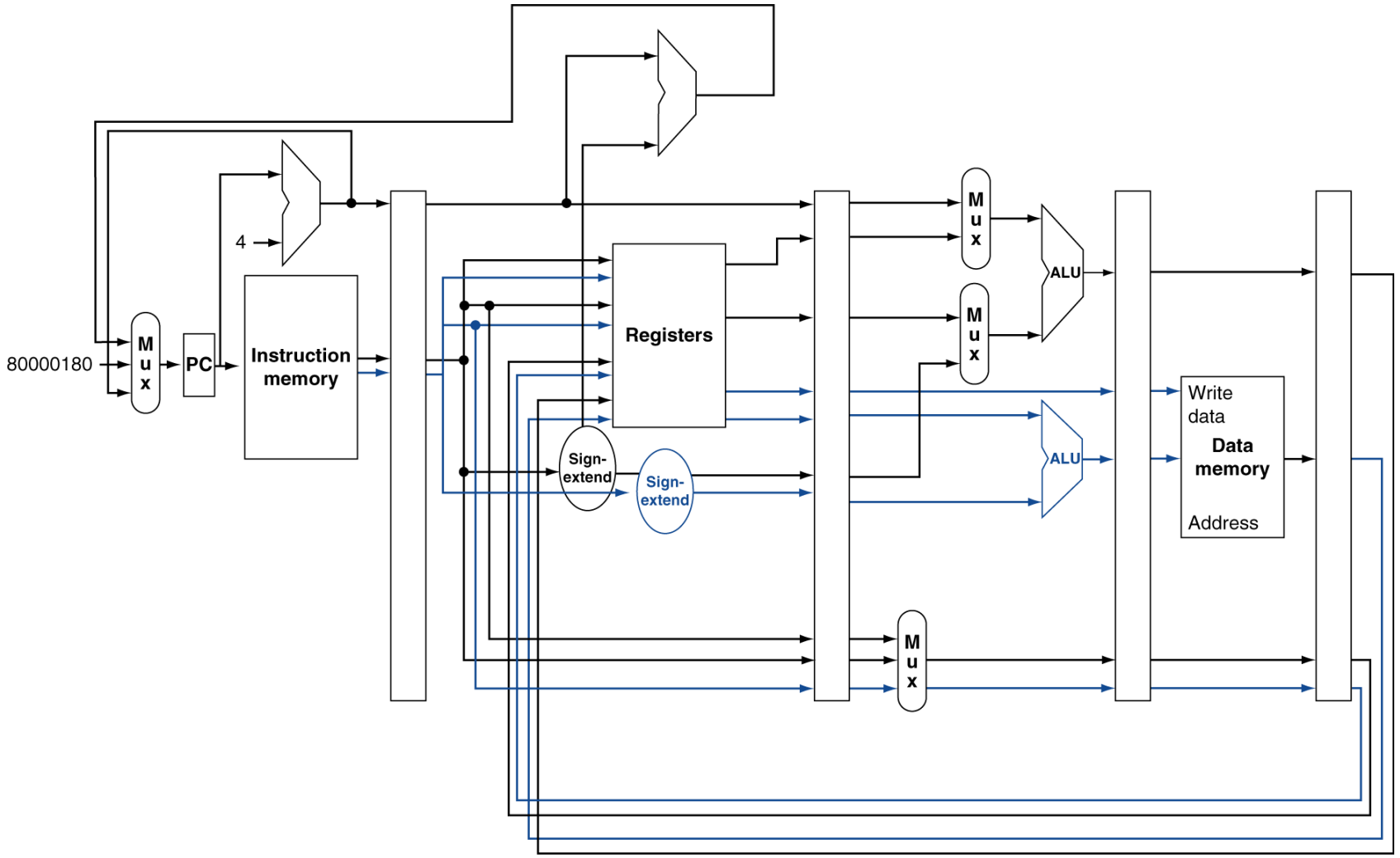
- **Static multiple issue**
  - The *compiler* groups instructions to be issued together
    - Instructions are grouped into **issue packets**
  - The *compiler* detects and avoids hazards
  - Also known as **very long instruction word (VLIW)**
- **Dynamic multiple issue**
  - The *hardware* examines the instruction stream and identifies instructions to issue each cycle
    - The compiler can help by reordering instructions
  - The *hardware* detects and avoids hazards
  - Also known as **superscalar**

MIPS with Static Dual Issue

- Compiler groups instructions into issue packets
  - Each packet is two instructions
    - 64-bit size, 64-bit alignment
  - First is an ALU/branch instruction
  - Second is a load/store instruction
  - Unused instruction are replaced with nop

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

MIPS with Static Dual Issue



Scheduling Example

- Schedule the following code for a dual-issue MIPS pipeline

```
Loop: lw   $t0, 0($s1)
      addu $t1, $t2, $s2
      sw   $t3, 0($s1)
      addi $s1, $s1, -4
      bne  $s3, $zero, Loop
```

	ALU/branch	Load/store	cycle
Loop:	addu $t1, $t2, $s2	lw $t0, 0($s1)	1
	addi $s1, $s1, -4	sw $t3, 0($s1)	2
	bne $s3, $zero, Loop	nop	3

Scheduling Example

- Schedule the following code for a dual-issue MIPS pipeline

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1,-4
      bne  $s1, $zero, Loop
```

	ALU/branch	Load/store	cycle
Loop:	nop	lw $t0, 0($s1)	1
	addi $s1, $s1,-4	nop	2
	addu $t0, $t0, $s2	nop	3
	bne $s1, $zero, Loop	sw $t0, 4($s1)	4

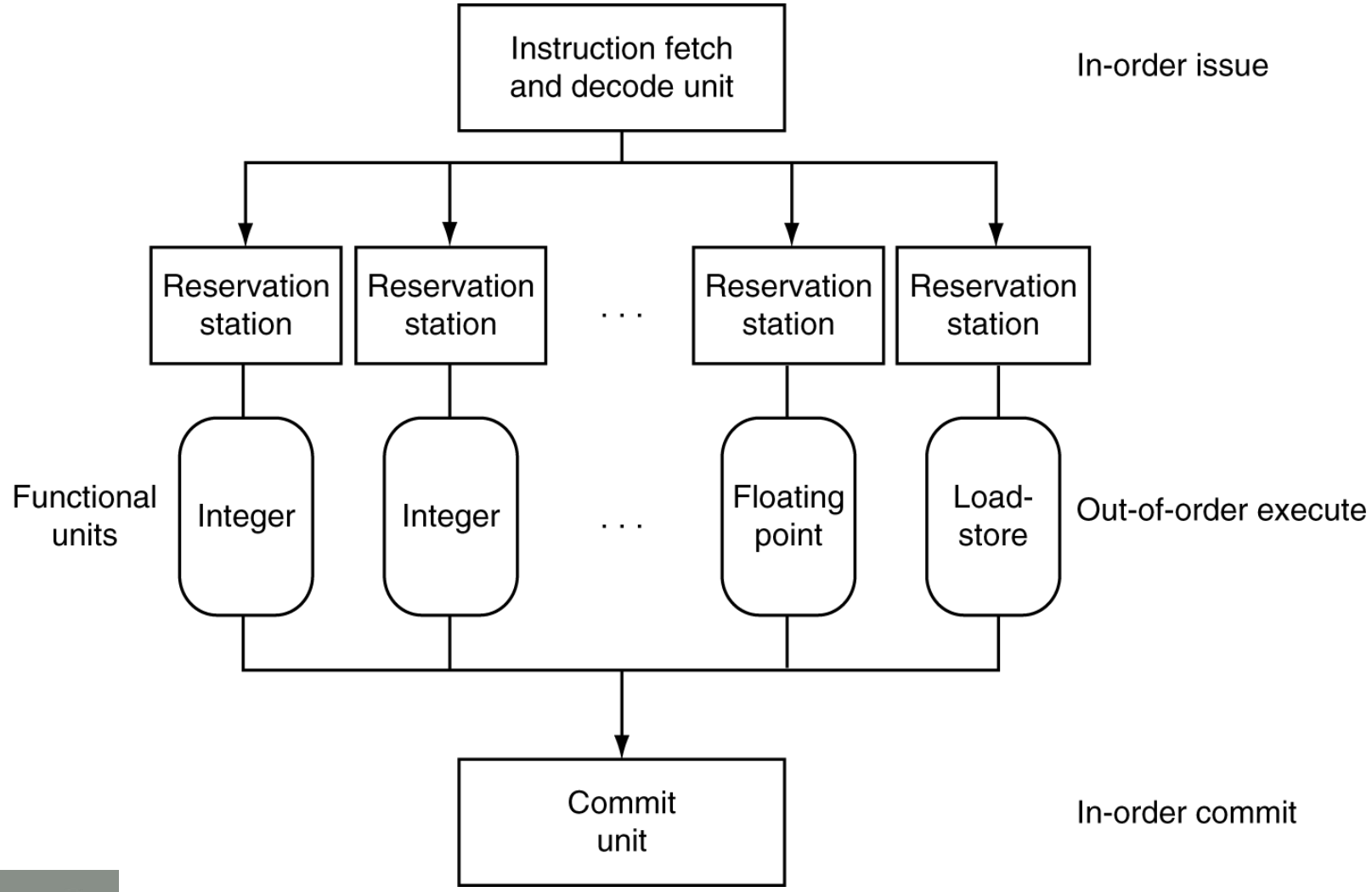
Dynamic Multiple Issue

- Hardware decides which instructions to issue every cycle
- Often combined with **dynamic pipeline scheduling**
  - Execute instructions out of order to avoid stalling
    - But commit result to registers in order
    - Also known as **out-of-order execution**
  - Example:

```
lw    $t0, 20($s2)
addu  $t1, $t0, $t2
sub   $s4, $s4, $t3
slti  $t5, $s4, 20
```

    - Can start `sub` while `addu` is waiting for `lw`

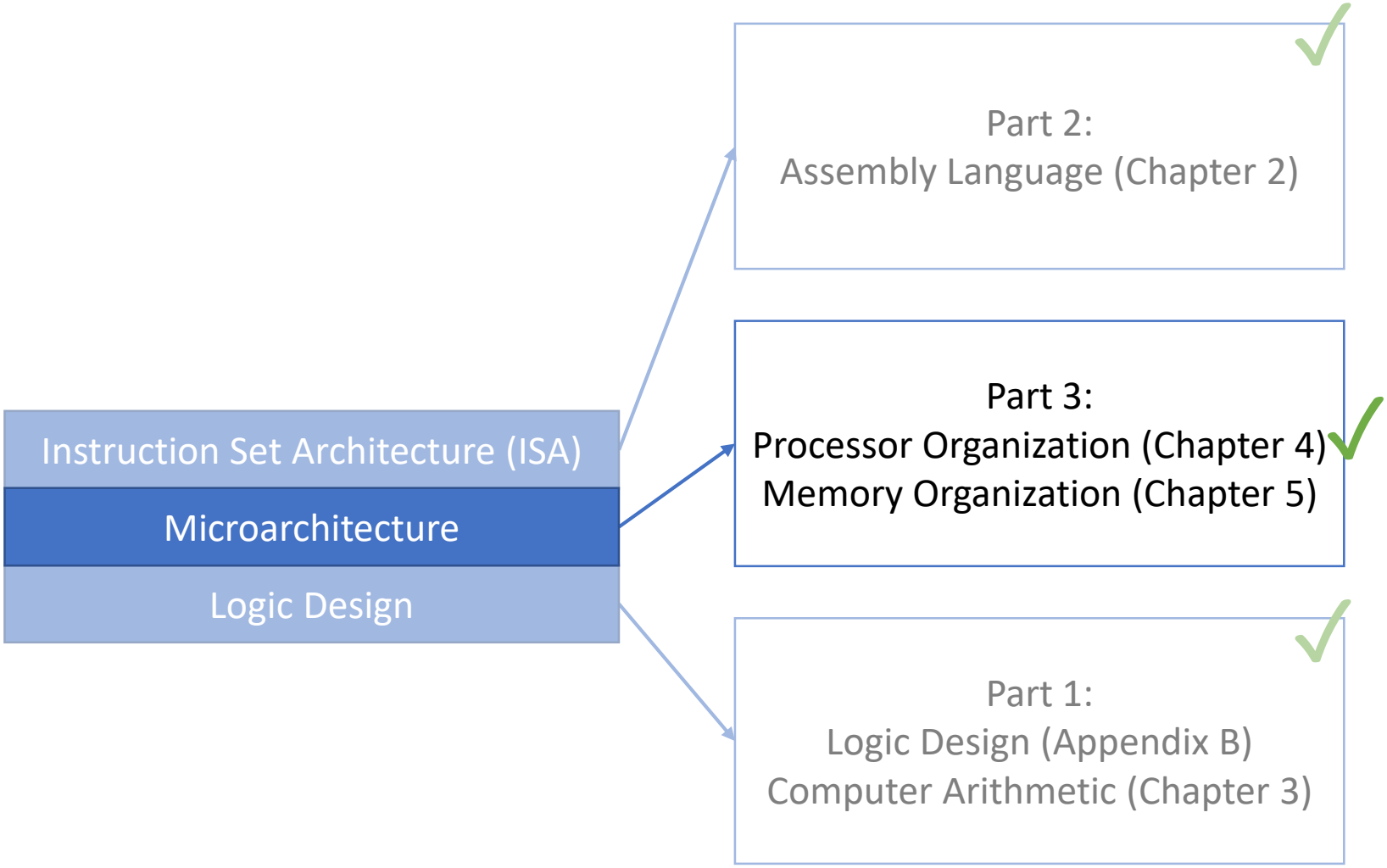
Dynamic Scheduling



Wide Pipelines Tradeoffs

- Advantage: wide pipelines provide speedup by extracting more ILP
- Disadvantage: wide pipelines bring far away instructions closer together, which increases the chances of hazards and stalls if the code has insufficient ILP
  - We will see how to fill these stalls later
  - Typical pipeline width is 4 in a modern processor

End of Processor Organization



Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 4.10